

SOFTWARE REQUIREMENTS SPECIFICATION

Computer & Electronics E-commerce Platform

Document Version	1.2
Source Document	Product Requirements Document (PRD) v1.2
Frontend	Next.js (App Router), React, Tailwind CSS
Backend	Node.js (Express or NestJS)
Database	PostgreSQL
Document Type	Software Requirements Specification (SRS)

Prepared from the provided PRD. This SRS preserves the requirements, terminology, architecture decisions, workflows, and open questions stated in that source.

1. Introduction

This Software Requirements Specification defines the functional, data, interface, architecture, and non-functional requirements for the Computer & Electronics E-commerce Platform. The platform covers computers, laptops, PC components, gadgets, electronics, Smart Home and Smart Office equipment, guided builders, laptop discovery, home-visit technician services, customer accounts, and a full administrative backoffice.

1.1 Purpose

The purpose of the system is to provide a scalable and maintainable e-commerce application with a dynamic category-driven product and filter model, generalized builder flows, complete customer/order operations, and centralized administration.

1.2 Scope

- PC and component catalog, including Laptop, Processor, CPU Cooler, Motherboard, Graphics Card, RAM, Power Supply, HDD, SSD, Casing, Monitor, Phone, TV, and related categories.
- Smart Home and Smart Office equipment under a shared Home Appliances category tree.
- Dynamic category-specific attributes and faceted search/filtering.
- PC Builder with hard compatibility validation; Smart Home and Smart Office Builders with guided step selection.
- Laptop Finder guided Q&A and ranked recommendations.
- Home-visit technician booking for PC repair and Smart Home/Office setup, decoration, and repair.
- Cart, compare, checkout, wishlist, orders, reviews, Q&A, support tickets, and customer dashboard.
- Admin modules for catalog, inventory, purchasing, selling, customers, delivery, marketing, support, team/RBAC, reports, and system settings.

1.3 Product Objectives

- Rebuild the site-map into a scalable, maintainable web application.
- Provide dynamic, per-category filters whose options change with the selected category/sub-category.
- Implement a generalized, data-driven Builder engine for PC, Smart Home, and Smart Office.
- Provide Laptop Finder recommendations using the same filter engine.
- Provide customer home-service booking and booking status management.
- Provide a complete customer account and administration ecosystem.

2. System Architecture & Technology

Layer	Technology / Requirement
Frontend	Next.js (App Router), React, Tailwind CSS
State / Data Fetching	React Query / SWR + Server Components
Backend	Node.js (Express or NestJS)
Database	PostgreSQL (relational + JSONB for dynamic attributes)
Cache	Redis (filter facets, session, cart)
Search	Elasticsearch / Meiliseach / Typesense; Meiliseach is recommended in the source for simpler
Authentication	JWT + refresh tokens; NextAuth optional on frontend
Media	S3-compatible storage for product images
Payments	SSLCommerz / bKash / Nagad / Card gateway, as applicable in BD market
Admin	Separate Next.js admin app or same app with role-based routes

Architecture flow: Next.js SSR/ISR pages communicate through REST or GraphQL with the Node.js API. The API uses PostgreSQL for core data, Redis for caching, and Meiliseach/Elasticsearch for search and facets.

Performance approach: Category and product listing pages use ISR; filter results are fetched client-side through the API once a filter is applied. Search handles faceted filtering and live option counts.

Backend recommendation: NestJS is recommended over plain Express because the category/attribute/filter logic becomes complex and modules such as Products, Categories, Attributes, Search, Orders, and PCBuilder can remain organized.

3. Core Dynamic Attribute & Filter Requirements

The central requirement is that every category can have its own filter set. The system must not use a single fixed filter schema for all categories.

3.1 Data Model

- categories: id, name, slug, parent_id for nested categories.
- attributes: id, name, type (select | range | multiselect | boolean), unit.
- category_attributes: id, category_id, attribute_id, is_filterable, display_order.
- attribute_options: id, attribute_id, value.
- products: id, category_id, name, price, offer_price, stock, brand_id, etc.
- product_attribute_values: id, product_id, attribute_id, value, or attribute_option_id for select type.

3.2 Required Behavior

- Admin attaches a different attribute set to each category.
- Product create/edit forms automatically render only the selected category's attributes.
- Category sidebars automatically render only filterable attributes attached to that category.
- Global filters Price Range and Availability are available on every category.

- Adding new categories or new attribute combinations must not require category-specific filter code.

3.3 Search Index

- Products and product_attribute_values are synchronized into a flattened search document.
- The search document contains product identity, category, price, and nested/flattened dynamic attributes.
- The search engine provides facets and live counts for filter options.
- Frontend filter UI is generated from category_attributes and populated with search-engine facet counts.

4. Functional Requirements

FR-01 — Home Page

- Header shall provide Logo, global Search with autocomplete, Categories mega menu, Builder dropdown, Home Appliances dropdown, Home Service, Offers, Happy Hour countdown, and Account.
- Mega Menu shall be data-driven from the nested categories table and support PC/component plus Smart Home/Smart Office branches.
- Home page shall support CMS-managed banner/sub-banner carousel and scrolling announcement marquee.
- Home page shall provide three Builder entry cards: PC, Smart Home, Smart Office.
- Home page shall provide Laptop Finder entry, Home Service promotion, Raise a Complaint / Servicing Center links, featured categories, ads slots, and a 20-card featured product grid.
- Footer shall contain support links, About, social links, app links, and copyright.

FR-02 — Catalog & Category Pages

- System shall support nested categories and sub-categories.
- Listing pages shall use dynamic category-specific filters.
- Sort and page-size controls shall be available; page sizes include 20/40/60.
- Product cards shall show image, name, short description, price, offer price, Buy Now, and Compare.

FR-03 — Product Details

- Product detail shall include gallery, name, price, regular price, stock status, product code, and brand.
- Product detail shall provide key features, payment options, ads slot, specifications table, description, Q&A, reviews, similar products, Share, Save/Wishlist, and Compare.
- Specifications shall map to product_attribute_values.

FR-04 — PC Builder

- Builder shall be a data-driven wizard with ordered steps.
- PC Builder shall enforce hard compatibility rules.
- CPU shall be selected first; Motherboard shall be filtered by CPU socket.
- RAM shall be filtered by motherboard RAM type and capped by slot count.
- Storage shall support multiple selections and respect SATA/M.2 port limits.
- Graphics Card, PSU, Casing, Casing Cooler, and CPU Cooler shall apply the stated compatibility/wattage/form-factor/socket constraints.
- Peripherals including Monitor, Keyboard, Mouse, Speaker, Headphone, WiFi/LAN Adapter, Anti-Virus, and UPS are optional and have no compatibility constraint.
- A persistent summary shall show selections and running total price.
- Add Full Build to Cart shall become available once all required steps are satisfied and shall add the build as one bundle line-item.

FR-05 — Smart Home Builder

- Smart Home Builder shall use the same generic builder engine as PC Builder.

- Default suggested order: Smart Hub, Smart Lighting, Smart Plugs/Switches, Smart Locks/Security, Cameras/Sensors, Laptop, Desktop, Other Smart Devices.
- No hard compatibility blocking shall apply by default.
- If hub_required does not match the chosen hub protocol, the system shall show a soft warning rather than block the selection.
- Laptop and Desktop are optional steps.

FR-06 — Smart Office Builder

- Default suggested order: Office Automation Hub, Smart Desk/Furniture, Conferencing Equipment, Office Lighting/Climate, Networking Equipment, Laptop, Desktop, Other Office Equipment.
- The same generic step, selection, summary, and cart mechanics shall apply.
- Laptop and Desktop are optional steps.

FR-07 — Home Service Booking

- Customers shall be able to book PC Repair (Home Visit), Smart Home Setup, Smart Home Repair, Smart Office Setup, and Smart Office Repair.
- Booking flow: select service type → enter/select address → choose date and time slot → add issue notes → confirm.
- Technicians shall be Employee records with a dedicated role such as Field Technician.
- Booking statuses shall follow requested → confirmed → technician_assigned → in_progress → completed; cancelled is reachable before in_progress.
- Customers shall view bookings in My Service Bookings and admin shall manage them in Service Bookings.

FR-08 — Laptop Finder

- Wizard shall collect Budget, Purpose, Screen Size, Portability Importance, and Other Features.
- Answers shall map to laptop filter query parameters.
- Final combined query shall be sent to the laptop product endpoint and results ranked/recommended.

FR-09 — Offers & Happy Hour

- Offers shall be CMS-managed and view-only, with links to products/categories.
- Happy Hour shall support start_time, end_time, discount_type (percent|fixed), discount_value, and linked products/categories.
- A scheduled job such as cron/BullMQ shall activate/deactivate pricing automatically.
- Frontend shall show a live countdown.

FR-10 — Cart, Compare & Checkout

- Cart shall support add, update, remove, and price recalculation.
- Offers and Happy Hour pricing shall be applied during recalculation.
- Compare shall show side-by-side specification tables using dynamic product attributes.
- Checkout shall support address selection, payment method selection, and order confirmation.

FR-11 — Customer Dashboard

- Login/Register shall use JWT and may optionally use OTP for the Bangladesh market.

- Customer shall manage order history/tracking, multiple addresses, own reviews, wishlist, profile settings, and support tickets.
- Customer shall view service bookings and may reschedule/cancel before technician assignment.

FR-12 — Admin / Backoffice

- Admin shall manage the complete e-commerce operation through catalog, inventory, buying/selling, customer-facing operations, reports, marketing, content/support/team, system tools, and settings.
- Categories shall include nested tree management and an Attribute Builder controlling dynamic filters.
- Products shall be CRUD-enabled and dynamically render category attributes.
- Inventory shall show live stock levels; Stock Movements shall record purchase-in, sale-out, return-in, damage-out, and manual adjustment.
- Purchases shall create supplier purchase orders and increase inventory.
- Orders shall support customer order management and status pipeline.
- Returns & Damages shall provide RMA workflow and damaged-stock write-off.
- Customers, Delivery, Coupons, Reviews, Transactions, Stock Report, Profit & Loss, Purchase History, Flash Sales, Newsletter, Banners, Content, Tickets, Employees, Roles, Activity Logs, System Tools, Profile, and system Settings shall be supported.

5. Administrative Requirements

Catalog & Inventory

Module	Requirement
Categories	Nested tree + Attribute Builder; controls dynamic filters.
Brands	Brand management.
Products	CRUD; auto-renders attribute fields per category.
Inventory	Live stock levels per product/warehouse.
Stock Movements	In/out log: purchase-in, sale-out, return-in, damage-out, manual adjustment.
Suppliers	Vendor directory linked to Purchases.

Buying & Selling

Module	Requirement
Purchases	Purchase orders from suppliers; increases inventory.
Orders	Customer order management and status pipeline.
Returns & Damages	RMA workflow + damaged-stock write-off; inventory adjustment.

Customer-Facing Operations

Module	Requirement
Customers	Customer directory/profiles and admin-side account view.
Delivery	Courier/shipping management and delivery status tracking.
Coupons	Discount codes with redemption; separate from Offers/Happy Hour.
Reviews	Moderation queue.
Transactions	Payment transaction ledger and refund tracking.

Reports

Module	Requirement
Stock Report	Stock reporting.
Profit & Loss	Profit and loss reporting.
Purchase History	Purchase history reporting.

Marketing

Module	Requirement
Flash Sales	Time-boxed promotion; likely maps to Happy Hour.
Newsletter	Email campaign management.
Banners	Home/sub-banner CMS control.

Content, Support & Team

Module	Requirement
Content	CMS pages such as About Us, policies, and static pages.

Module	Requirement
Tickets	Support ticket queue linked to customer Raise a Complaint.
Employees	Staff accounts.
Roles	RBAC permission sets per role.
Activity Logs	Audit trail of admin actions.
System Tools	Cache clear, search reindex, maintenance mode, etc.
Profile	Logged-in admin account settings.

Settings

Module	Requirement
Site	Store identity: name, logo, contact info, SEO defaults.
Currency	Base currency and formatting.
Email	SMTP/transactional email provider configuration.
SMS	SMS gateway configuration for OTP and notifications.
Payment	Gateway credentials/configuration for bKash, Nagad, SSLCommerz, card.
COD	Availability by area and min/max order value.
Pusher	Real-time notification service configuration for live order/stock alerts.

6. Data Requirements

The following entities and relationships are explicitly specified by the source for the dynamic catalog and builder model.

Entity	Required Fields / Role
Category	id, name, slug, parent_id; supports nested category tree.
Attribute	id, name, type (select/range/multiselect/boolean), unit.
CategoryAttribute	id, category_id, attribute_id, is_filterable, display_order.
AttributeOption	id, attribute_id, value.
Product	id, category_id, name, price, offer_price, stock, brand_id, etc.
ProductAttributeValue	id, product_id, attribute_id, value or attribute_option_id.
BuilderType	Data-driven builder type such as PC Builder, Smart Home Builder, Smart Office Builder.
BuilderStep	Ordered step linked to a category; allowsMultiple and isRequired determine behavior.
ServiceType	Supported home-service types.
ServiceBooking	Customer booking record and status pipeline.
Employee	Staff record; may carry Field Technician role.

6.1 Builder Data Model

- A BuilderType owns an ordered list of BuilderSteps.
- Each BuilderStep points to a category and defines whether the step allows multiple selection and whether it is required.
- The same engine must support PC Builder, Smart Home Builder, and Smart Office Builder; a future builder type can be added by configuration rather than code changes.
- PC Builder uses hard compatibility rules; Smart Home and Smart Office use guided selection with soft warnings where applicable.

7. API Requirements

Endpoint	Purpose
GET /api/categories/:slug/filters	Return filterable attributes for a category, available options, and counts.
GET /api/categories/:slug/products?...filters	Return filtered product list and updated facet counts.

Frontend implementation shall use /app/category/[slug]/page.tsx as the category page server component. A reusable FilterSidebar shall render controls from dynamicAttributes: select → checkbox list, range → price/number slider, boolean → toggle. URL query parameters shall drive filter state for shareability, bookmarkability, and SSR-friendly navigation.

8. Business Rules & Workflows

8.1 PC Builder Compatibility

- CPU must be selected before motherboard.
- Motherboard compatibility is determined by CPU socket.
- RAM compatibility is determined by motherboard RAM type and slot capacity.
- Storage selection is constrained by available SATA/M.2 ports.
- PSU must provide sufficient wattage; insufficient wattage is a hard block.
- Casing must match the required form factor.
- CPU Cooler must match the CPU socket.
- Peripherals are optional and do not participate in compatibility blocking.

8.2 Smart Home / Smart Office Compatibility

- These builders do not hard-block compatibility by default.
- When `hub_required` does not match an already selected hub protocol, the system displays a soft warning.
- Builder steps are configurable and can include optional Laptop/Desktop steps.

8.3 Home Service Status

- requested → confirmed → technician_assigned → in_progress → completed.
- cancelled may be reached before in_progress.
- Customer dashboard displays current booking status.
- Admin manages bookings and technician assignment.

8.4 Promotion Pricing

- Offers are separate from code-based Coupons.
- Happy Hour is time-boxed and linked to products/categories.
- Scheduled processing activates/deactivates Happy Hour pricing.
- Cart price recalculation applies applicable offers and Happy Hour pricing.

9. UI / UX Requirements

- The global navigation shall expose Categories, Builders, Home Appliances, Home Service, Offers, Happy Hour, and Account.
- Category filters shall be generated dynamically and change according to the selected category/sub-category.
- Builder screens shall present ordered step cards, selection states, and a persistent summary with running total.
- Product pages shall expose gallery, commercial information, dynamic specifications, reviews, Q&A, similar products, wishlist, compare, and share actions.
- Admin UI shall expose the confirmed sidebar module structure and role-based access.

10. Non-Functional Requirements

ID	Area	Requirement
NFR-01	SEO	Server-rendered category/product pages, clean slugs, and per-category SEO description b
NFR-02	Performance	ISR for catalog pages, Redis caching for facet counts, CDN for images.
NFR-03	Scalability	Attribute-driven schema so new categories can be added through admin configuration with
NFR-04	Security	Input validation, rate limiting on search/API, and RBAC for admin.

11. Development & Delivery Phases

Phase	Scope
1	DB schema (categories, attributes, products), Admin category + attribute builder, basic auth.
2	Product CRUD + dynamic specification rendering, category listing + dynamic filter sidebar.
3	Search engine integration (Meilisearch), product details, cart, compare, wishlist.
4	PC Builder rules engine, Laptop Finder wizard.
5	Checkout + payment gateway, Offers/Happy Hour engine.
6	Customer dashboard (orders, tickets, reviews, addresses), Q&A/Review system.
7	Ads/CMS blocks, SEO polish, sitemap, category SEO description block, performance, caching, QA + launch.

12. Acceptance-Oriented Requirements Summary

- A category administrator can create a category, attach attributes, and configure filterability/order without developer changes.
- A product form displays only the attributes configured for its category.
- A category page displays only that category's configured filters plus global Price Range and Availability.
- Search results return updated facet counts after filtering.
- PC Builder prevents incompatible component combinations according to the stated rules.
- Smart Home and Smart Office builders warn on hub/protocol mismatch without hard blocking by default.
- Customers can complete checkout, manage orders, and use wishlist/compare/reviews/support features.
- Customers can create and track home-service bookings through the defined status pipeline.
- Administrators can manage catalog, stock, purchases, orders, returns, customers, delivery, promotions, transactions, reports, CMS, support, staff/RBAC, audit logs, and system settings.

13. Open Questions Requiring Stakeholder Confirmation

- Which payment gateways are required for the Bangladesh market: bKash, Nagad, SSLCommerz, card, or a combination?
- Should Compare be cross-category or restricted to products within the same category?
- Should Happy Hour discounts stack with regular Offers, or should they be mutually exclusive?

Source note: This SRS was generated from the provided Product Requirements Document (PRD) v1.2 and does not add requirements from external sources. The source references ADMIN.md, DATABASE.md, and RULES.md for additional field-level/workflow detail, but those files were not included in the supplied material; therefore this SRS does not invent those unspecified details.